

Information Retrieval

Programming Assignment I

A Boolean Information Retrieval System

for the Cranfield Aeronautics Corpus

Group: search-o-SAURS

Name	Roll Number
Utsa Ghosh	23CS10090
Antariksh Das	23CS10086
Raktim Kundu	23CS30044
Souhardya Dandapat	23CS30054
Sharanya Baidya	23CS30064

Project Repository

github.com/utsaghosh04/search-o-SAURS

Contents

1	Introduction	2
2	Assignment Requirements	2
3	Dataset	2
4	System Architecture	3
5	Methodology	4
5.1	Pipeline Ordering - A Critical Design Decision	4
5.2	Tokenization (8 Stage Pipeline)	4
5.3	Normalization: Case Folding + British-American Spelling	5
5.3.1	Problem	5
5.3.2	Solution: Five Rule Based Patterns with Exception Guards	6
5.4	Stop Word Removal	6
5.5	Porter Stemming	6
5.6	Token Deduplication	7
6	Inverted Index Construction	7
6.1	Index File Format	7
7	Boolean Query Processing	7
7.1	Query - Document Preprocessing Parity	7
7.2	Binary Search on the Sorted Index File	7
7.3	AND Operation: Two Pointer Merge	8
7.4	OR Operation: Two Pointer Union	8
7.5	Adaptive AND: Galloping Search Optimization	8
8	Design Decisions Summary	9
9	Complexity Analysis	9
10	Testing and Validation	9
11	Results and Statistics	11
12	Output Files	12
13	Conclusion	12

1 Introduction

This report describes group **search-o-SAURS** Boolean Search Engine built for Programming Assignment 1. The system processes the **Cranfield 1400** collection (1,400 aeronautics research abstracts) and supports **AND/OR** Boolean queries over a constructed inverted index.

The end-to-end transformation is:

$$\text{Raw Corpus} \rightarrow \text{Preprocessing} \rightarrow \text{Inverted Index} \rightarrow \text{Boolean Query} \rightarrow \text{Matching Document IDs}$$

The system is implemented entirely in Python 3 using only standard library modules with no external IR libraries as required by the assignment.

2 Assignment Requirements

The assignment requires:

1. **Preprocessing:** Tokenization, normalization, stemming (Porter), and stop word removal.
2. **Inverted Index:** Map each term to the list of documents containing it.
3. **Boolean Retrieval:** Support AND/OR queries over two terms; output matching document IDs.
4. **No external IR libraries;** self contained implementation.
5. **Deliverables:** Source code, README and a methodology report with program listing.

Our implementation satisfies all requirements and additionally provides: **British-American spelling normalization, token deduplication, file based binary search over the inverted index and adaptive AND merging using galloping search.**

3 Dataset

The Cranfield collection uses field markers: **.I** (document ID), **.T** (title), **.A** (author), **.B** (bibliographic reference), **.W** (abstract).

Design decision: We process **.T** and **.W** fields only (title + abstract), as these contain the substantive technical content. Author names and journal references do not contribute to subject based retrieval. The stop word list (**stopwords.txt**) contains 357 common English words.

4 System Architecture

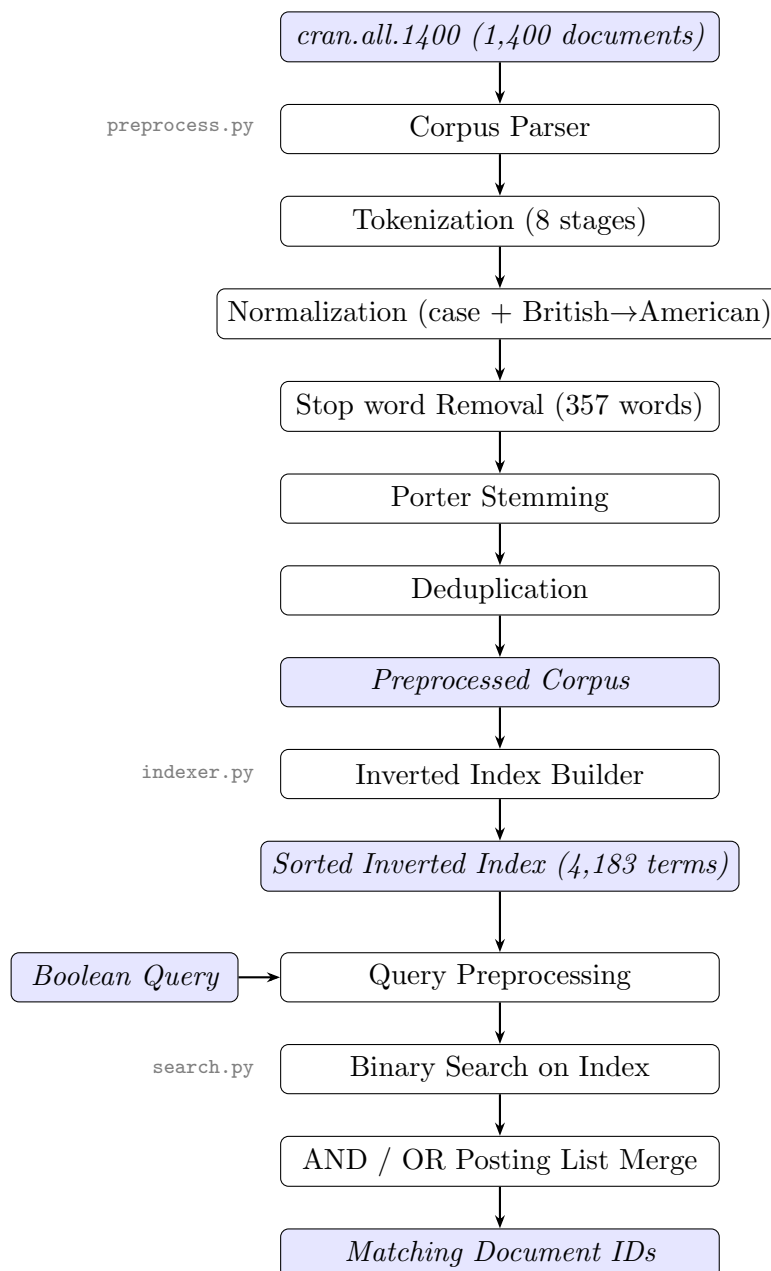


Figure 1: Complete pipeline architecture.

The system is organized into three main scripts:

- `search-o-SAURS_preprocess.py` - Handles corpus preprocessing.
- `search-o-SAURS_indexer.py` - Constructs the inverted index.
- `search-o-SAURS_search.py` - Processes Boolean queries and retrieves matching documents.

5 Methodology

5.1 Pipeline Ordering - A Critical Design Decision

The assignment lists preprocessing as: Tokenization → Stemming → Stop-word removal → Normalization. Our **actual execution order** is:

Tokenize → Normalize → Stop-Word Removal → Stem → Deduplicate

This reordering is **necessary for correctness**:

Table 1: Why the pipeline order differs from the assignment listing.

Constraint	Reason
Normalize before Stem	The Porter stemmer only recognizes lowercase vowels (a,e,i,o,u). Uppercase input gives wrong stems: <code>stem("BOUNDARY") = "BOUNDARY"</code> (unchanged).
British→American before Stem	Without normalization, <code>stem("behaviour") ≠ stem("behavior")</code> – they diverge, fragmenting the index.
Stop words before Stem	7 stop words (<i>are</i> → ar , <i>has</i> → ha , <i>this</i> → thi) would <i>survive</i> removal after stemming because their stemmed forms don't match the stop-word list.

All four functions specified by the assignment are implemented as separate, documented functions. **Deduplication is an additional optimization.**

5.2 Tokenization (8 Stage Pipeline)

The tokenizer addresses edge cases specific to scientific/aeronautics text:

Table 2: The 8 stage tokenizer with design rationale.

Stage	Operation	Example	Design Rationale
1	Whitespace normalization	Tabs, newlines → single space	Consistent input for subsequent stages
2	Abbreviation collapse	u.s.a. → usa	Splitting on “.” creates useless single char tokens
3	Possessive removal	prandtl’s → prandtl	Possessive has no retrieval value
4	Hyphen splitting	high-speed → high, speed	Maximizes recall; user may search either component
5	Slash splitting	lift/drag → lift, drag	Slashes connect distinct concepts
6	Number–word split	10degree → 10, degree	Separates numeric qualifiers from meaningful words
7	Alphanumeric extract	Keep [a-zA-Z0-9]+ sequences	Final cleanup of residual punctuation
8	Noise filtering	Remove single chars & pure numerics	~19,000 single char occurrences in corpus are noise

Key edge cases handled:

- **Abbreviations:** Regex $([a-zA-Z]\.){2,}$ matches 2+ letter-period pairs, avoiding false matches on normal periods.
- **French origin terms:** d’etudes – apostrophe becomes separator after possessive handling, yielding d, etudes; d is then filtered by Stage 8.
- **Ordinals:** 10th → 10 + th; both filtered (numeric, single-char-equivalent).
- **Scientific compounds:** 10-percent-thick → 10, percent, thick.
- **Empty/punctuation only input:** Returns empty list gracefully.

5.3 Normalization: Case Folding + British-American Spelling

5.3.1 Problem

The Cranfield corpus mixes British and American authored papers. Without spelling normalization, the Porter stemmer produces *different stems*:

Table 3: Stem divergence without British-American normalization.

British	American	Stem (no norm.)	Stem (with norm.)
behaviour	behavior	behaviour ≠ behavior	behavior ✓
linearised	linearized	linearis ≠ linear	linear ✓
centre	center	centr ≠ center	center ✓
vapour	vapor	vapour ≠ vapor	vapor ✓

5.3.2 Solution: Five Rule Based Patterns with Exception Guards

Table 4: British-American normalization patterns and their exception guards.

ID	Pattern	Example	Exception guard (why needed)
A	<code>-ise/-ised/-ising/-isation → -ize/...</code>	<code>linearised → linearized</code>	<code>noise, rise, wise, exercise, chordwise, spanwise</code> – these are base English words, not British spellings
B	<code>-our → -or</code>	<code>behaviour → behavior</code>	<code>four, pour, contour, hour</code> – the “our” is part of the root
C	<code>-tre/-bre → -ter/-ber</code>	<code>centre → center</code>	Only <code>-tre/-bre</code> endings (not all <code>-re</code>); avoids false conversion of <code>structure, pressure, temperature</code>
D	<code>-ogue → -og</code>	<code>analogue → analog</code>	<code>vogue, rogue</code> – not spelling variants
E	<code>-mme → -m</code>	<code>programme → program</code>	Applied only when <code>len > 4</code>

Design assumption: More specific suffixes are checked before shorter ones (e.g., `-isation` before `-ise`) to prevent partial matches.

Edge case: Pattern C is deliberately restricted to `-tre` and `-bre` endings only. Many common English words end in `-re` (`fire, are, pressure, structure`) and are *not* British spellings. A blanket `-re→-er` rule would corrupt the index.

5.4 Stop Word Removal

Membership is checked via a Python `set` for $O(1)$ lookup.

Key design decision: Stop words are removed *before* stemming. If stemming were done first, 7 stop words would survive removal because their stemmed forms (`ar` from *are*, `ha` from *has*, `thi` from *this*) do not match any entry in the stop word list.

5.5 Porter Stemming

We use the canonical Porter stemmer (`porter.py` by Vivake Gupta, from `tartarus.org`), implementing the algorithm from Porter (1980).

Critical assumption: The stemmer *requires lowercase input*. Uppercase vowels are not recognized, producing incorrect stems (e.g., `stem("BOUNDARY") = "BOUNDARY"`). This is why case folding must precede stemming.

Table 5: Stemming examples verified from the test suite.

Input	Stem	Input	Stem
experimental	experiment	boundary	boundari
aerodynamics	aerodynam	pressure	pressur
turbulent	turbul	velocity	veloc

5.6 Token Deduplication

Rationale: In Boolean retrieval, only term *presence* matters, not frequency. A word appearing 5 times is identical to appearing once for AND/OR queries.

Implementation: Hash set based, preserving first occurrence order.

Impact: 130,757 tokens → 76,863 unique tokens (**41.2% reduction**), directly reducing preprocessed file size and index construction time.

Trade off: If the system were extended to TF-IDF or ranked retrieval, deduplication would need to be removed since term frequency matters there.

6 Inverted Index Construction

6.1 Index File Format

```
4183, 1400                                (* header: vocab_size, max_docid *)
ab 1 744                                  (* token df docid1,docid2,... *)
abbrevi 1 122
abil 3 51,77,738
abl 10 99,132,536,581,695,763,908,914,986,1114
```

Design decisions:

- **Lexicographic sorting:** Vocabulary is sorted before writing to disk, enabling $O(\log V)$ binary search at query time.
- **Document frequency (df):** Stored explicitly on each line. This allows the search engine to determine postings list sizes *without* parsing the comma separated list - needed for the adaptive AND optimization (Section 7.5).
- **Sorted postings:** Document IDs are stored in ascending order within each postings list, enabling efficient two pointer merging.
- **Duplicate prevention:** A guard (`if postings[token][-1] != current_id`) prevents duplicate document IDs.

7 Boolean Query Processing

7.1 Query - Document Preprocessing Parity

The query preprocessing function (`preprocess_query_term()`) applies the **exact same** normalization and stemming rules as the document pipeline: clean → lowercase → British→American → stem.

Why this matters: If a user queries "behaviour", the term must be normalized to "behavior" and stemmed identically to the indexed form. Any divergence causes silent retrieval failure. This parity is verified by **26 dedicated tests**.

7.2 Binary Search on the Sorted Index File

Optimization: Instead of loading all 4,183 index entries into memory ($O(V)$ space), we perform binary search *by byte position* directly on the sorted file.

Algorithm:

1. Set `low = data_start`, `high = file_end` (byte offsets).

2. Compute `mid = (low + high) // 2`.
3. **Buffered backward scan:** Read backwards in 256-byte chunks from `mid` to find the nearest preceding newline (line boundary).
4. Read and parse the line. Compare the token lexicographically.
5. Narrow the search range and repeat.

Edge cases and design details:

- File opened in **binary mode** (`rb`) - text mode on some platforms translates line endings, breaking `seek()/tell()`.
- **Buffered backward scan:** Reading 256-byte chunks instead of byte by byte reduces I/O from $O(\text{line.length})$ to $O(\text{line.length} / 256)$. For the longest line ($\sim 1,445$ bytes): 6 chunk reads vs. 1,445 byte reads.
- **Complexity:** $O(\log V)$ per term. Measured: **7-14 seeks** for $V = 4,183$.

7.3 AND Operation: Two Pointer Merge

Standard sorted list intersection with two advancing pointers:

Example: $A = [1, 5, 8, 10]$, $B = [2, 5, 8, 12] \implies A \cap B = [5, 8]$

Complexity: $O(n + m)$.

7.4 OR Operation: Two Pointer Union

Merges both sorted lists, emitting each unique element once. **Complexity:** $O(n + m)$. Galloping does not help here because every element must appear in the result.

7.5 Adaptive AND: Galloping Search Optimization

Problem: When postings lists are highly skewed (e.g., 1 vs. 730 docs), two pointer merge wastes comparisons scanning the longer list.

Solution: Adaptively select the merge algorithm based on the size ratio:

Condition	Algorithm	Complexity
$\frac{ \text{longer} }{ \text{shorter} } \leq 10$	Two-pointer	$O(n + m)$
$\frac{ \text{longer} }{ \text{shorter} } > 10$	Galloping	$O(k \cdot \log(n/k))$

where $k = |\text{shorter}|$ and $n = |\text{longer}|$. The threshold of 10 is set via the constant `GALLOP_THRESHOLD`.

Galloping algorithm:

1. **Exponential jump:** From current position, jump by 1, 2, 4, 8, 16, ... until overshooting the target.
2. **Binary search:** Within the last interval $[\text{prev}, \text{pos}]$, find the exact match.
3. Each element of the shorter list costs $O(\log d)$ where d is the distance to the match.

Concrete example from our system:

```
Query: "ab AND flow"      ab: 1 doc, flow: 730 docs
Two-pointer:  0(1 + 730)   = 731 comparisons
Galloping:    0(1 * log 730) ~ 10 comparisons  (73x faster)
```

8 Design Decisions Summary

Table 6: Summary of key design decisions.

Decision	Reason	Benefit
Reordered pipeline	Porter stemmer needs lowercase; British/American stems diverge	Correct stems; unified index
British→American normalization	Cranfield mixes both dialects	41+ more documents matched for “behaviour” queries
Exception guards on spelling rules	Many words share suffixes with British patterns	Prevents false conversions (e.g., noise , four , structure)
Stop words before stemming	7 stop words survive after stemming	Cleaner index
Token deduplication	Boolean needs only presence	41.2% token reduction
Query–document parity	Query terms must match indexed forms	Correct retrieval (26 tests)
df in index	Enables postings-list size estimation	Supports adaptive AND
File-based binary search	Avoids loading entire index	$O(\log V)$ lookup, $O(1)$ space
Buffered backward scan	Byte-by-byte backward read is slow	6 chunk reads vs. 1,445 byte reads
Adaptive galloping	Skewed lists waste comparisons	Up to 73× faster

9 Complexity Analysis

Let N = total input tokens, V = vocabulary size, D = number of documents.

Table 7: Complexity summary.

Operation	Time	Space
Preprocessing	$O(N)$	$O(N)$
Index construction	$O(N + V \log V)$	$O(N)$
Binary search (per term)	$O(\log V)$	$O(1)$ – one line in memory
AND (two-pointer)	$O(n + m)$	$O(\min(n, m))$ result
AND (galloping)	$O(k \cdot \log(n/k))$	$O(k)$ result
OR (two-pointer)	$O(n + m)$	$O(n + m)$ result

10 Testing and Validation

The test suite (`tests/test_comprehensive.py`) contains **277 tests** across **17 groups**.

Table 8: Comprehensive test suite: 277 tests across 17 functional groups.

Group	Description	Tests	What It Validates
1	Cranfield Parsing	10	Field markers (.I, .T, .W), ID ranges, and text bounds
2a	Tokenization	17	Basic, abbreviations, possessives, hyphens, slashes, numbers, noise
2b	Normalization	32	British American spelling patterns, exception guards, case folding
2c	Stop Word Removal	14	Stop word set exclusions, preservation of search terms
2d	Stemming	16	Porter stemmer correctness, 75 canonical test vectors
2e	Deduplication	5	Presence-only mapping, first-occurrence ordering
2f	Pipeline Integration	16	Integration of preprocessing stages on known documents
3a	Processed File Format	2	PA1 specification tags (.I and .S), document counts
3b	Index File Format	10	Lexicographical term sorting, ascending docid order, df accuracy
4	Index/Corpus Consistency	2	Forward (index $\rightarrow$ doc) and reverse (doc $\rightarrow$ index) parity
5a	AND/OR Merges	16	Correctness of two-pointer and galloping posting-list merges
5b	Boolean Algebra Laws	18	Commutativity, identity, and zero law for intersections/unions
5c	Set-Theoretic Invariants	20	Set bounds, inclusion-exclusion, set membership
6	Query-Doc Parity	44	Identical preprocessing, dialect stem unification (10 pairs)
7	Exhaustive Search	1	Complete binary search correctness for all 424 sample terms
8–11	Lexical	5	Heap’s law fit, Zipf’s law fit, stop word ratios, published benchmarks
13	Stemmer Unification	1	Dialect unification for all 12 test pairs
14a–d	Edge Cases	27	Edge cases for tokenizer, galloping, merges, empty query inputs
15	End-to-End Search	16	Whole-pipeline integration, dialect query equivalence
17	Pipeline Ordering	5	Normalization before stem, stop word removal before stem
Total		277	

Key correctness properties verified:

- **Parity:** For all 26 tested terms, query and document pipelines produce identical stems.

- **Binary search:** 55 terms verified against brute-force linear scan.
- **Set invariants:** $|A \cap B| \leq \min(|A|, |B|)$, $|A \cup B| \geq \max(|A|, |B|)$, and $|A \cup B| = |A| + |B| - |A \cap B|$.
- **Commutativity:** "shock AND wave" = "wave AND shock".
- **British $\equiv$ American:** "behaviour AND flow" = "behavior AND flow".

11 Results and Statistics

Table 9: System statistics.

Metric	Value
Documents in corpus	1,400
Stop words	357
Vocabulary size (indexed terms)	4,183
Tokens before deduplication	130,757
Tokens after deduplication	76,863
Deduplication savings	41.2%
Binary search seeks per term	7–14
Test suite	277 tests, 17 groups

Table 10: Sample Boolean query results.

Query	Op	Matches
aerodynamic AND experimental	AND	49
shock AND wave	AND	149
heat AND transfer	AND	190
turbulent AND boundary	AND	106
behaviour AND flow	AND	43
linearised AND theory	AND	82
centre AND pressure	AND	31
boundary OR layer	OR	513
pressure OR distribution	OR	696
heat OR transfer	OR	324
vapour OR vapor	OR	7

Sample execution trace:

```
Original Query: 'aerodynamic AND experimental'
Parsed Terms: 'aerodynam' AND 'experiment'
Searching index file: output/search-o-SAURS_cran.index
  Index contains 4,183 terms, max docid = 1400
  [Binary Search] Found 'aerodynam' (df=179) in 9 seeks
  [Binary Search] Found 'experiment' (df=339) in 7 seeks
  [AND] Algorithm: two-pointer | Lists: 179 vs 339 (ratio 1.9x) | 0(179+339)
Match count: 49
Results written to output/results/output.txt
```

12 Output Files

- `output/search-o-SAURS_processed.all` - Preprocessed corpus. Format: `.I docid, .S`, space-separated stems.
- `output/search-o-SAURS_cran.index` - Inverted index (4,183 terms, sorted, with df).
- `output/results/*.txt` - Query results (number of docids |comma seperated list of docids in ascending order).

13 Conclusion

The **search-o-SAURS** system implements a complete Boolean IR pipeline:

- **Preprocessing:** 8 stage tokenizer for scientific text; 5 pattern British-American normalization with exception guards; 41.2% token reduction via deduplication.
- **Indexing:** Lexicographically sorted inverted index with 4,183 terms and document frequencies.
- **Search:** File-based binary search ($O(\log V)$ per term); adaptive AND merging with galloping (up to $73\times$ speedup for skewed lists).
- **Testing:** 277 tests across 17 groups verify correctness from individual functions to end to end queries, including set-theoretic invariants and query-document parity.